

ISAD – Topic 1

Introduction to Information System Analysis
and Design

LEARNING OBJECTIVES

1. Understand Software Development & Information System

2. Explain SDLC overview

3. Identify System Analyst role

4. Feasibility Analysis and Project Planning

WHAT IS ISAD?

- Information System Analysis and Design studies how to understand organizational problems and plan suitable information systems.
- Analysis focuses on what users and the organization need; design focuses on how the solution will be structured.
- ISAD is not only technical work; it combines business understanding, user communication, documentation, and modeling.
- Key idea: good systems begin with clear analysis before detailed design and implementation.

WHAT IS AN INFORMATION SYSTEM

- A system that collects, processes, and outputs information
- It is not only a computer program; it also includes users, rules, and business activities
- Supports decision making
- Works together to:
 - Collect → Process → Output information

Examples: a university registration system stores student data, checks course rules, and shows schedules

WHAT IS SOFTWARE DEVELOPMENT

- Software development is the complete work of creating software from idea to real use.
- It includes planning, understanding the problem, designing the solution, coding, testing, releasing, and improving the product later
- Software development is not only writing code; it also includes teamwork, communication, and quality control
- Examples: building a food delivery app from first concept to daily operation.

MAIN COMPONENTS OF AN INFORMATION SYSTEM

- People: users, managers, support staff, and technical teams who interact with the system.
- Processes: business rules and work steps that transform inputs into useful outputs.
- Data: facts captured, stored, processed, and reported by the system.
- Technology: software, database, hardware, network, and integration services.
- Controls: rules that protect data quality, access rights, security, and compliance.

WHY SOFTWARE DEVELOPMENT NEEDS A PROCESS

- A process gives order to the work so the team knows what to do first and what comes next.
- Without a process, teams may miss requirements, waste money, or deliver systems that users do not want.
- A clear process improves planning, communication, and quality.
- Key words: process, sequence, quality, coordination, control.

SDLC Overview

Planning

Analysis

Design

Implementation

Testing

Deployment

Maintenance

MEANING OF SDLC

- SDLC means System Development Life Cycle. It is a structured set of stages used to create and improve information systems.
- The word life cycle shows that a system has a beginning, growth, change, and maintenance over time.
- Different organizations name the stages in different ways, but the main idea is the same.
- Example stages: planning, analysis, design, implementation, testing, deployment, maintenance.

WHY SDLC MATTERS?

- Avoid failure
- Improve quality
- Manage complexity
- The SDLC helps teams break a large project into smaller and more manageable parts.
- It supports better budgeting, scheduling, risk control, and review at each stage.
- It also helps stakeholders understand progress and approve important decisions.

The Planning Stage

- Planning is the early stage where the team identifies the problem, business need, goal, and project scope.
- The team asks basic questions: Why do we need this system? Who will use it? What result is expected?
- At this stage, the team also estimates cost, time, and resources.

Example: a hospital plans a patient record system to reduce paperwork and speed up service

The Analysis Stage

- Analysis is the stage where the team studies the current situation and gathers detailed requirements.
- The goal is to understand what the new system must do before deciding how to build it.
- Common activities include interviews, observation, document study, and model creation.
- Key words: requirement, current system, user need, business rule

The Design Stage

- Design turns requirements into a technical plan for building the system.
- The team decides the architecture, database structure, interfaces, controls, and detailed component behavior.
- Good design connects business needs to practical technical choices.

Example: deciding what tables, screens, and services an online banking system should have

The Implementation Stage

- Implementation is the stage where the system is actually built and configured.
- Programmers write code, database specialists create schemas, and teams integrate different modules.
- Implementation follows the design documents and should not ignore user requirements.
- Key words: coding, integration, configuration, build

The Testing Stage

- Testing checks whether the system works correctly and whether it meets user requirements.
- Teams test functions, data accuracy, security, performance, and user experience.
- Finding errors before release is much cheaper than fixing them after deployment.
- Example: testing whether an order system calculates total cost and tax correctly.

The Deployment Stage

- Deployment means moving the finished system into the real user environment.
- This may include installation, data migration, user training, and changeover from the old system to the new one.
- A deployment plan reduces confusion and service interruption.
- Key words: release, migration, training, conversion.

The Maintenance Stage

- Maintenance is the work done after the system is released and used in real life.
- It includes fixing bugs, improving features, adapting to new rules, and supporting users.
- Many systems spend most of their life in maintenance, not in first-time development.
- Example: updating payroll software when tax rules change.

SDLC PHASES AND MAIN DELIVERABLES

- Planning: business need, initial scope, feasibility result, project plan, and approval decision.
- Analysis: current-system findings, stakeholders, business rules, well-formed requirements, use cases, DFD, ERD and UML analysis models.
- Design: high-level architecture, low-level design, database design, UI/UX wireframes, reports, controls, and validation rules.
- Implementation: developed modules, source code, configured database, integrated functions, and technical/user documentation.
- Testing: test cases, defect log, verified functions, data validation results, and acceptance results.
- Deployment: release plan, installation or migration, user training, changeover plan, and go-live support.
- Maintenance: bug fixes, enhancements, user support, and adaptation to new business rules or needs

Common SDLC Flow

Waterfall Model

V-Shaped Model

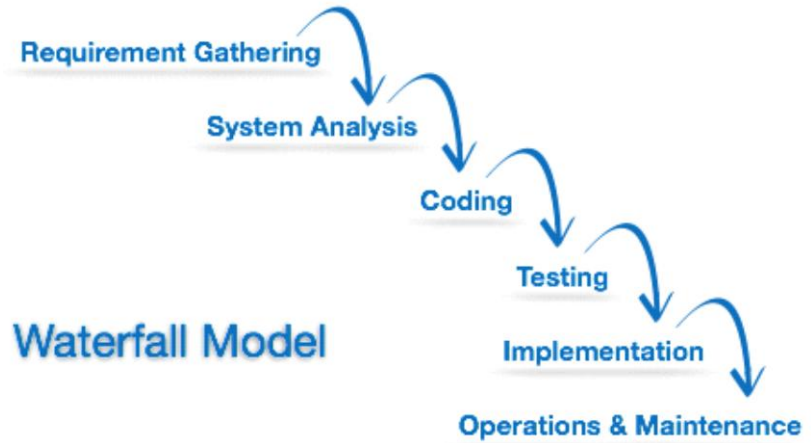
Spiral Model

Iterative Model

Incremental Model

Agile Model – Scrum Process

WaterFall Model

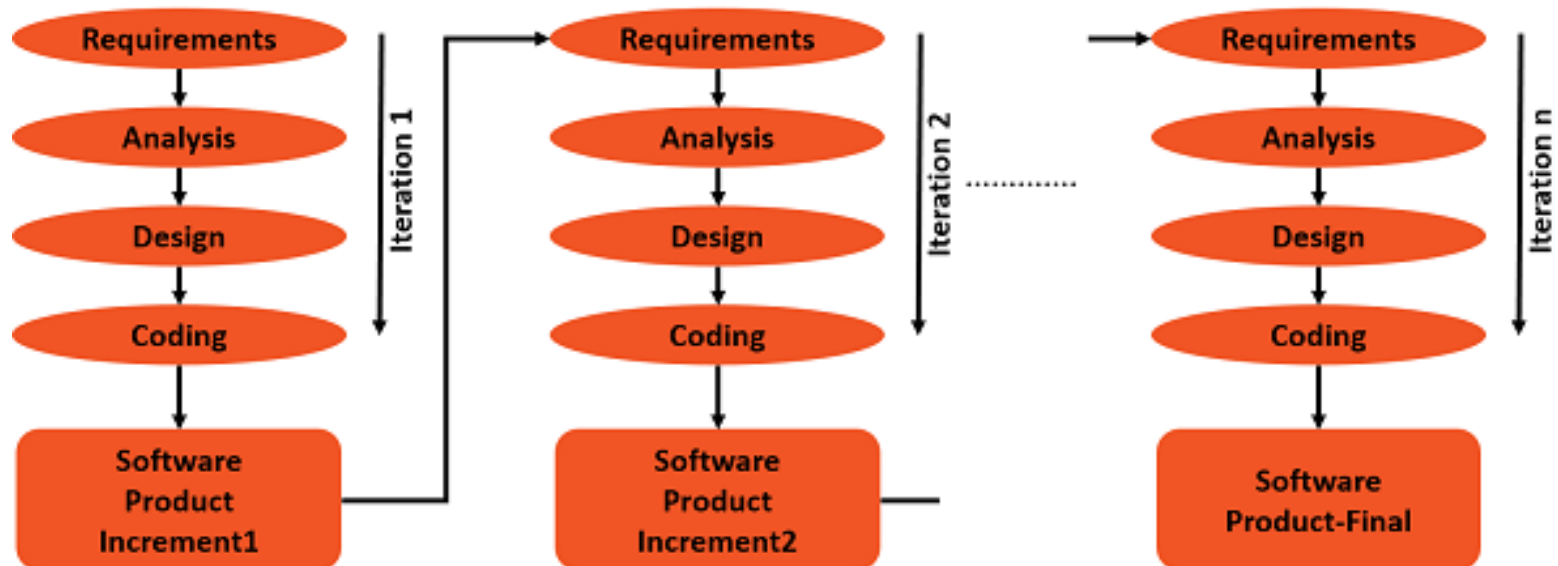


- The waterfall model is a traditional SDLC approach where work moves step by step in order.
- It is easy to understand and useful when requirements are stable and well known early.
- Its weakness is low flexibility when changes happen later.

Example: a government reporting system with fixed rules may use a waterfall-like process.

Iterative Model

- The iterative model builds the system in repeated cycles. Each cycle improves the product little by little.
- This helps teams learn from feedback and reduce the risk of building the wrong thing.
- It is useful when not all requirements are clear at the beginning.
- Key words: cycle, iteration, feedback, refinement.





Agile Way of Working

- Agile is a flexible way of developing software in small parts, with regular feedback from users and stakeholders.
- Teams often work in short time boxes called sprints and deliver useful pieces often.
- Agile supports change, but it still needs discipline, communication, and clear priorities.

Example: a startup improving its app every two weeks based on customer comments.



Compare Waterfall and Agile

Criteria	Waterfall	Agile
Approach	Full planning at the beginning	Short cycles (iterations)
Flexibility	Low (hard to change)	High (easy to adapt)
Best for	Stable requirements	Changing or unclear requirements
Main focus	Predictability	Flexibility
Development style	Step-by-step (sequential)	Incremental & iterative



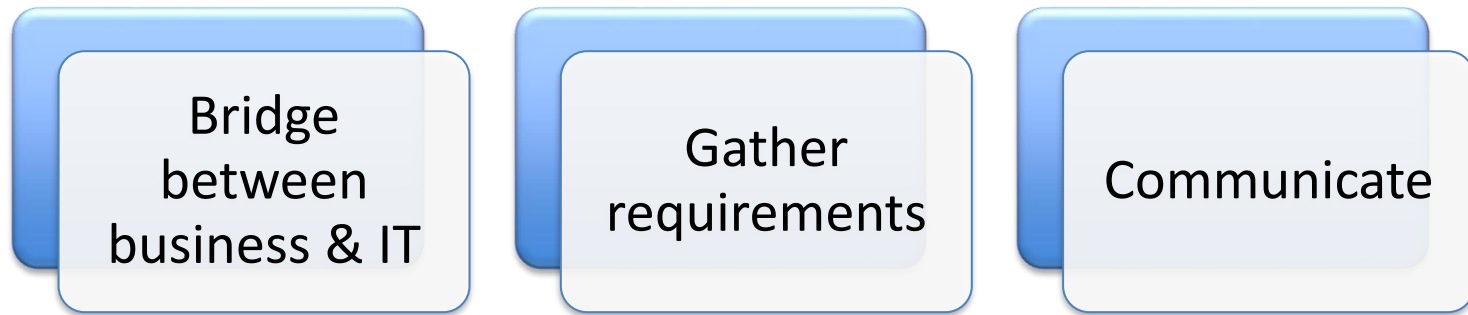
SCRUM AS AN AGILE EXAMPLE

- Scrum organizes work into short iterations called sprints, usually producing a usable product increment.
- Main roles: Product Owner, Scrum Master, and Development Team.
- Main artifacts: Product Backlog, Sprint Backlog, and Product Increment.
- For ISAD, Scrum reminds analysts that requirements and design can be refined through feedback, not frozen forever.

Choosing an SDLC Approach

- Teams choose an approach based on project size, requirement clarity, regulation level, budget, risk, and team experience.
- For safety-critical systems, a more controlled process may be needed. For fast-changing products, iterative work may be better.
- The process should match the problem, not just follow fashion.
- Example: a banking core system and a student club app do not need the same approach.

ROLE OF SYSTEM ANALYST



Analyst decides **WHAT** to build,

Developer decides **HOW**

Main Responsibility of a SA

- The analyst studies the current system, identifies problems, and documents user needs clearly.
- The analyst also prepares models, supports feasibility study, helps define scope, and checks whether the design matches requirements.
- In many projects, the analyst joins meetings, reviews documents, and helps manage change.
- Example: writing use cases and process diagrams for an e-commerce system.



Analyst as a Problem Solver

- A systems analyst does not only record what users say; the analyst also looks for the real problem behind the request.
- Sometimes users ask for a feature, but the deeper need is speed, accuracy, security, or better reporting.
- Good analysis means solving the right problem, not only the visible symptom.
- Example: users ask for more staff, but the real issue is a slow manual workflow.

Analyst as a Communicator

- Analysts must speak clearly with different groups, such as end users, managers, developers, testers, and database designers.
- They translate business language into technical requirements and technical limits into simple business language.
- Good communication reduces mistakes and conflict.
- Key words: translation, discussion, meeting, clarification.

Analyst and Programmer

Criteria	Systems Analyst	Programmer
Main role	Analyze requirements & plan solutions	Write and test code
Key question	“What should the system do?”	“How do I build it?”
Focus	Understanding business needs	Implementing technical solutions
Skills	Analysis, communication, problem-solving	Coding, debugging, technical skills
In small teams	May also do programming	May also do analysis

Analyst and Project Manager

Criteria	Systems Analyst	Project Manager
Main role	Analyze requirements & design solutions	Manage project execution
Focus	Requirements, business logic, solution quality	Schedule, cost, team coordination
Key question	“What should the system do?”	“When and how will it be delivered?”
Responsibilities	Gather requirements, create models, refine use cases	Plan timeline, track progress, manage resources
Example	Refines use cases and system functions	Tracks deadlines and project status
Relationship	Supports technical clarity	Ensures project delivery

Technical Skills for Analysts

◆ Core Knowledge

- Database & data structure
- System architecture & networks
- Interfaces (UI/API)
- Modeling tools (UML, diagrams)

◆ Coding Level

- No deep coding required
- Understand logic & technical constraints

◆ Why It Matters

- Create realistic solutions
- Communicate with developers
- Improve analysis & design quality

Business Skills for Systems Analysts

◆ Business Understanding

- Business goals & strategies
- Policies & workflows
- Performance measures (KPIs)
- User needs & pain points

◆ Key Thinking

- How does the system create **value**?
- How does it improve **efficiency**?
- How does it support **decision making**?

◆ Why It Matters

- Avoid building systems that are technically correct but **not useful**
- Ensure solutions align with real business needs
- ◆ Example: a sales system should support business targets, not just store customer names

People Skills for Analysts

◆ Core Skills

- Listening & questioning
- Teamwork & collaboration
- Negotiation & presentation
- Empathy (understanding users)

◆ Key Challenge

- Users may be busy, unsure, or unclear
- Analysts must guide conversations effectively

◆ Why It Matters

- Build trust with stakeholders
- Collect accurate and complete information
- Improve communication across teams

Stakeholders in a System Project

◆ Who are Stakeholders?

- People or groups who **affect** or are **affected by** the system

◆ Examples

- End users
- Managers & owners
- Customers
- Developers
- Auditors & support staff

◆ Key Challenge

- Different stakeholders have different **interests & expectations**
- Analysts must **balance needs and priorities**

◆ Why It Matters

- Ensure the system meets real needs
- Avoid conflicts and misunderstandings

Example of Stakeholder Differences

◆ Different Needs

- End users → speed & simple interfaces
- Managers → reports & control features
- Security staff → strict access rules
- Customer service → fast access to accounts

◆ Real Example

- Hospital system:
 - Doctors → medical data
 - Nurses → patient care info
 - Billing staff → payment details
 - Patients → personal records

◆ Key Challenge

- Stakeholders want different things → potential conflicts
- Analysts must identify differences **early**

What Is Feasibility Analysis?

◆ What is it?

- An initial evaluation to determine if a system idea is **viable and worth investing in**

◆ What Does It Check?

- Can it be built with current technology? (*technical*)
- Is it cost-effective? (*economic*)
- Will it work in real operations? (*operational*)
- Can it be completed on time? (*schedule*)
- Does it fit the organization? (*organizational*)

◆ Why It's Important

- Good ideas are not always **achievable**
- Helps avoid wasting time, money, and resources

Operational Feasibility

◆ What Is It?

- Evaluates whether the system will **work effectively in real operations** and be **accepted by users**

◆ What Does It Consider?

- Fit with existing workflows
- User training needs
- Policies and procedures
- User readiness and acceptance

◆ Why It Matters

- A technically strong system can still **fail** if:
 - Users do not trust it
 - Or cannot use it effectively

◆ Example

- Staff may resist a new inventory system if it **adds too many steps**

Technical Feasibility

◆ What Is It?

- Evaluates whether the required **technology, skills, and infrastructure** are available

◆ What Does It Consider?

- Hardware and software platforms
- Network capacity
- Security requirements
- Technical complexity

◆ Why It Matters

- Project risk increases if technology is **too advanced** for the organization
- Ensures the system can be built with available resources

◆ Example

- A company wants AI features but lacks **data quality and skilled engineers**

Economic Feasibility

◆ What Is It?

- Evaluates whether the **expected benefits outweigh the costs**

◆ What Does It Consider?

- Costs: software, hardware, salaries, training, maintenance, change management
- Benefits: time savings, fewer errors, better service, increased revenue

◆ Why It Matters

- Ensures the project delivers **real value within budget**
- Helps decide if the investment is worthwhile

Schedule Feasibility

◆ What Is It?

- Evaluates whether the project can be **completed within the available time**

◆ What Does It Consider?

- Task estimates and workload
- Dependencies between tasks
- Potential risks and delays

◆ Why It Matters

- Even valuable projects can **fail** if deadlines are unrealistic
- Helps set achievable timelines

◆ Example

- A tax reporting system must be ready **before the next financial year**

Organizational & Legal Feasibility

◆ What Is It?

- Evaluates whether the project aligns with **organizational structure, policies, and legal requirements**

◆ What Does It Consider?

- Management support and approval
- Fit with company policies and culture
- Legal regulations and compliance (privacy, security, industry laws)
- Risk of non-compliance and penalties

◆ Why It Matters

- A system may fail if it **conflicts with internal rules or external laws**
- Ensures the project is acceptable, compliant, and sustainable

◆ Example

- A healthcare system must comply with **patient data privacy regulations**

PROJECT PLANNING

◆ What Is It?

- The process of organizing **tasks, people, timeline, and resources** to execute a project

◆ Key Focus

- What needs to be done? → Tasks
- Who will do it? → Roles
- When will it happen? → Schedule
- What is needed? → Resources

◆ Why It Matters

- Turns ideas & feasibility into a **clear action plan**
- Helps manage risks and control progress

◆ Example

Plan tasks, budget, roles, and milestones for a student management system

Project Scope

- ◆ What Is It?

Defines what is **included and excluded** in the project

- ◆ Key Focus

- Clear boundaries of features and functions
- Written in **clear, testable terms**

- ◆ Why It Matters

- Prevents **scope creep** (uncontrolled expansion)
- Keeps project on time, budget, and focus

- ◆ Example

Phase 1: student registration & payment

Not included: hostel management

Work Breakdown Structure

- ◆ What Is It?
 - Breaks a project into **smaller tasks and subtasks**
- ◆ Key Focus
 - Decomposition into manageable parts
 - Identify tasks, subtasks, and deliverables
- ◆ Why It Matters
 - Makes work easier to **estimate, assign, track, and review**
 - Turns complex projects into understandable structure

Scheduling & Gantt Charts

- ◆ What Is Scheduling?
 - Assigns tasks to a **timeline** with start and finish dates
- ◆ What Is a Gantt Chart?
 - A visual chart showing tasks as **time-based bars**
- ◆ Why It Matters
 - Shows task sequence and overlaps
 - Helps track progress and deadlines
- ◆ Example
 - Analysis: week 1–3
 - Design: week 4–6
 - Testing: week 9–10

Milestones & Deliverables

- ◆ What Is a Milestone?

- An important **checkpoint** in the project
- Example: requirement approval, prototype completion

- ◆ What Is a Deliverable?

- A concrete **output** produced by the project
- Example: report, diagrams, prototype, final system

- ◆ Key Difference

Milestones → measure **progress**

Deliverables → show **results/output**

Risk Management in Planning

- ◆ What Is Risk?

- A possible problem affecting **time, cost, quality, or scope**

- ◆ What Should Be Done?

- Identify major risks
- Estimate **probability & impact**
- Prepare **mitigation plans**

- ◆ Examples

- Unclear requirements
- Limited budget
- Staff changes
- Vendor delays
- Data migration failure

Resource & Communication Planning

Resources

- People, budget, tools, devices, software, time

Communication Planning

- Define how information is shared:
 - Meetings, updates, reports, approvals

Why It Matters

- Keeps stakeholders **informed and aligned**
- Reduces misunderstanding and delays

Example

- Weekly status meeting
- Monthly steering report
- Issue log review

Case Study

University Course Registration System

At a university, course registration is currently handled through a combination of spreadsheets, paper forms, email communication, and manual approval by academic staff. At the beginning of each semester, students must review course offerings, check prerequisites, complete registration forms, and wait for approval from the Academic Affairs Office.

Because the process is not centralized, students often face several problems:

- Course information is not always updated in time
- Class capacity is difficult to track accurately
- Prerequisite checking is done manually
- Schedule conflicts are sometimes detected too late
- Students may register for the wrong section or miss deadlines
- Staff spend too much time answering repeated questions and correcting errors

Lecturers and program coordinators also experience difficulties:

- They cannot easily monitor the number of enrolled students
- Class lists are often delayed
- Course changes are communicated inconsistently
- Workload planning becomes harder due to inaccurate enrollment data

University management wants to develop a **Course Registration System** that allows students to register online, supports automatic checking rules, reduces errors, and improves academic administration efficiency

Case Study

Project Goal

Develop an online **University Course Registration System** that:

- Helps students search and register for courses
- Checks prerequisites automatically
- Prevents schedule conflicts
- Controls class capacity
- Supports add/drop actions within deadlines
- Provides lecturers and staff with accurate enrollment information

Define:

1. Main business problem
2. Stakeholders
3. Evaluate the feasibility of this system in 4 areas:
 - Technical
 - Economic
 - Operational
 - Schedule

Case Study

Student Course Registration System

Problem

- Manual / fragmented registration
- Errors: schedule conflicts, missing courses
- Slow grade lookup
- System overload during peak time

Stakeholders

- Students
- Staff
- Lecturers
- Program coordinators

Feasibility

- **Operational:** High (users accept)
- **Technical:** Medium (need server upgrade)
- **Economic:** Positive (save time & cost)

Project Planning

- Define scope (registration, grades, timetable)
- Gather requirements (students & staff)
- Design database (students, courses, grades)
- Plan schedule & testing
- User training before launch

Value

- Improve efficiency & accuracy
- Align with user needs
- Connect theory → real-world system

Quiz

Qu No.	Content	Answer
1	What does SDLC stand for?	A. Software Design Life Cycle B. System Development Life Cycle C. System Design Logical Cycle D. Software Data Logic Control
2	Who decides <i>what</i> the system should do?	A. Programmer B. Tester C. Systems Analyst D. Project Manager
3	What is feasibility analysis used for?	A. Writing code B. Testing system C. Checking if project is practical D. Deploying system
4	What is WBS?	A. A testing method B. A database design C. Breaking project into tasks D. A programming tool
5	What does a milestone represent?	A. Final product B. A checkpoint in project C. A risk D. A requirement

Conclusion, Summary and Review

- Topic 1 provides a **foundation of system development**, including concepts, roles, feasibility, and planning before detailed design begins.
- The **SDLC** gives a structured process, while the **systems analyst bridges business needs and technical solutions**.
- **Feasibility analysis** ensures that a project is practical, realistic, and worth investing in before moving forward.
- **Project planning** organizes scope, tasks, schedule, resources, communication, and risks to guide successful execution.
- Understanding stakeholders and their differences helps avoid conflicts and build systems that truly meet user needs.